

ST0244 - Programming Languages Programming

School of Applied Sciences and Engineering - EAFIT University

Lecturer Alexander Narváez Berrío.

Feb 2026

PRACTICE I - STUDENT REGISTRATION SYSTEM (HASKELL + PROLOG).

VALUE: 15% OF THE FINAL COURSE GRADE

INSTRUCTIONS:

Develop two versions of the same program to manage student registration at a university:

1. Version A: Haskell (functional approach)
2. Version B: Prolog (logic programming approach)

Both versions must provide the same menu options and the same functional behavior: registering student entry/exit, searching by student ID, listing students loaded from a file, and calculating time spent at the university.

Functional Requirements:

1) Check In

The program must allow registering a student's entry to the university.

- Request the student's ID
- Record the current entry time (see note below about time)

2) Search by Student ID

The program must allow searching for a student currently inside the university by their ID.

- If the student is still inside, display their information
- If not found (or already checked out), show an appropriate message

3) Time Calculation

The program must calculate how long a student stayed in the university.

- Based on entry time and exit time
- Show the duration in a clear format (minutes, or HH:MM, etc.)

4) Students List (Load from File)

Choosing this option must:

- Load student information from University.txt
- Store it in a list
- Display the list in the terminal

5) Check Out

The program must allow checking out a student.

- Request the student's ID

ST0244 - Programming Languages Programming

School of Applied Sciences and Engineering - EAFIT University

Lecturer Alexander Narváez Berrío.

Feb 2026

- Record the current departure time
- Update the stored data accordingly

Technical Requirements:

A) Haskell Version (100% Haskell)

- Must be developed entirely in **Haskell**
- Use a **list** to store student information in memory during execution
- Implement **a function for each functional requirement**
- Implement **file management** for persistence
- At startup: **load University.txt** into the list
- On changes (check-in/check-out): **save** back to the file

B) Prolog Version (100% Prolog)

- Must be developed entirely in **Prolog**
- Use a **list** (or a list of structures/terms) to store student information in memory during execution
- Implement **a predicate for each functional requirement**
- Implement **file management** for persistence
- At startup: **load University.txt** into the list (or into facts derived from that list)
- On changes (check-in/check-out): **save** back to the file

Important Note about “Current Time”

Since Haskell and Prolog handle time differently depending on environment, you may implement “time” in one of these accepted ways (must be consistent across the project):

- **Option 1 (Recommended):** Use the system clock if available in your language setup.
- **Option 2:** Ask the user to input the time (HH:MM) and store it.
- **Option 3:** Store time as an integer (minutes since 00:00) for easy subtraction.

Supporting Material

The lecturer will provide:

- A code base
- An explanatory video

You must complete what is missing in order to fulfill the practice in both languages (Haskell and Prolog).

Presentation and Defense

The final presentation (sustentación) of the practice will be conducted exclusively in person during class with the lecturer.

There will be no video submission.

Each group/student must:

- Submit the GitHub repository as specified.
- Present and execute both implementations (Haskell and Prolog) live in class.

ST0244 - Programming Languages Programming

School of Applied Sciences and Engineering - EAFIT University

Lecturer Alexander Narváez Berrío.

Feb 2026

- Demonstrate all required functionalities.
- Answer questions related to the implementation, logic, file handling, and design decisions.

Assessment

The evaluation of the practice will be distributed as follows:

1) Correct Operation of Both Implementations – 30%

- Haskell version runs correctly.
- Prolog version runs correctly.
- All functional requirements are implemented.
- Proper file persistence is demonstrated.
- Clean and structured code.

2) In-Class Defense (Presencial) – 70%

During the in-class presentation, students will be evaluated on:

- Live execution of both programs.
- Demonstration of all required options:
 - Check In
 - Search by Student ID
 - Time Calculation
 - List Students (load from file)
 - Check Out
- Ability to explain:
 - Data structures used (lists)
 - File handling
 - Time calculation logic
 - Differences between functional (Haskell) and logic programming (Prolog)
- Ability to answer conceptual and technical questions from the lecturer.

⚠ The defense is mandatory and exclusively presencial.

Failure to attend the in-class defense will significantly affect the final of the practice grade.